

Systèmes d'exploitation I

Chapitre 6 : Introduction à la sécurité des OS

Moufida Maimour

TELECOM Nancy – 2ème année

2025-2026

- 1 Introduction - Rôle de l'OS dans la sécurité globale d'un SI
- 2 Système de protection - Mécanismes matériels
- 3 C.I.A Triade
- 4 Vulnérabilités classiques

Objectifs :

- Comprendre le rôle central du système d'exploitation dans la sécurité d'un ordinateur.
- Identifier les mécanismes matériels et logiciels assurant la protection des ressources.
- Reconnaître les principales menaces et failles.
- Découvrir la logique de l'investigation numérique et les bases de la forensic mémoire.

Rôle de l'OS dans la sécurité globale d'un SI

- ✓ L'OS occupe une position centrale dans la sécurité d'un SI.
- Il **contrôle, arbitre et protège l'accès** à toutes les ressources de la machine.

- **Ressources logicielles :**

- ▶ **contrôle d'accès** au système de fichiers (permissions, utilisateurs, groupes),
- ▶ **authentification** et gestion des comptes utilisateurs,
- ▶ **séparation des privilèges** root/user
- ▶ ...

- **Ressources matérielles :**

- ▶ processeur : gestion du **temps partagé** et prévention du monopole CPU,
- ▶ mémoire : **isolation** des espaces d'adressage des processus,
- ▶ périphériques d'E/S : accès contrôlé via le **mode privilégié**

- ✓ la protection est mise en œuvre par l'OS mais elle s'appuie sur des **mécanismes matériels** (MMU, registres de mode, interruptions, etc.) pour garantir l'isolation et le respect des privilèges.

Système de protection- Mode privilégié

✓ Un processeur moderne fournit 2 modes d'exécution minimum permettant la séparation des modes d'exécution: **utilisateur (user)** et **privilégié (kernel)**

→ **Appels système** pour l'exécution des instructions privilégiées

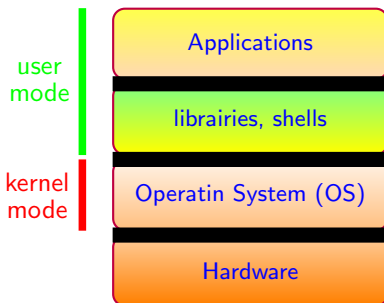
✓ L'ensemble des appels système (API du noyau) constitue l'interface entre les programmes utilisateurs et le noyau. **ex.** read(), write(), fork(), execve(), etc.

★ **ex.** Les instructions d'E/S

✓ Pour accéder à un périphériques d'E/S, un programme utilisateur doit passer par le noyau via un appel système :

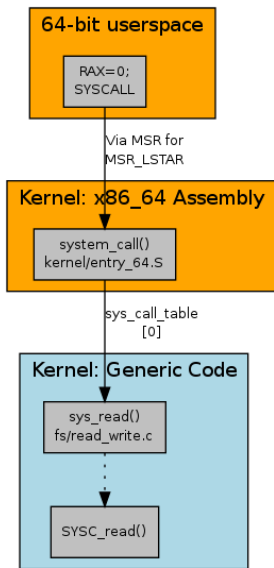
- une interruption est déclenchée via une instruction spéciale :
trap, syscall (x86-64), int 0x80 (x86)
→ **événement synchrone**
- les paramètres sont transmis via les registres, la pile, ou la mémoire

✓ Un appel système agit comme un appel de fonction dont le corps est **exécuté par le noyau** "pour le compte" du processus utilisateur



Système de protection

Appels système / x86-64



★ Cas Linux/x86-64

- numéro de l'appel système placé dans RAX
ex. 0 pour `read()`
- passage d'arguments (6 max) par registres : RDI, RSI, RDX, R10, R8, R9
ex. `read(fd, buf, count)` : RDI, RSI, RDX
- l'instruction `syscall` déclenche le passage en mode noyau

● Confidentialité

- ▶ *Principe* : empêcher la divulgation non autorisée des données
- ▶ *Menace typique* : fuite de données ex. lecture mémoire d'un autre processus
- ▶ *Mécanismes OS associés* : isolation mémoire, permissions de fichiers, etc.
- ▶ *Objectif* : seul l'utilisateur ou le processus autorisé peut accéder à une ressource donnée

● Intégrité

- ▶ *Principe* : empêcher la modification illégitime des données ou du système
- ▶ *Menace typique* : escalade de privilèges
- ▶ *Mécanismes OS associés* : contrôle d'accès, signatures, etc.
- ▶ *Objectif* : maintenir la cohérence et la fiabilité des données et du système

● Disponibilité

- ▶ *Principe* : garantir l'accès légitime et continu aux ressources
- ▶ *Menace typique* : déni de service - monopolisation de ressources (CPU, mémoire, ressources système)
- ▶ *Mécanismes OS associés* : quotas, priorités, gestion CPU/RAM, surveillance (watchdog)
- ▶ *Objectif* : empêcher qu'un utilisateur malveillant ou un bug ne bloque le système

Vulnérabilités classiques

1. Dénî de service

- **Principe** : rendre une ressource indisponible en la surchargeant.
- **Cibles typiques** : CPU (boucle infinie), mémoire (allocation massive), table de processus (fork bomb)

Déni de service - Exemple : UC

```
while(1) { ; }
```

→ Une **interruption** périodique (top d'horloge) permet à l'OS de reprendre la main et de répartir le temps processeur entre les processus (temps partagé).

Déni de service - Exemple : UC

```
while(1) { ; }
```

→ Une **interruption** périodique (top d'horloge) permet à l'OS de reprendre la main et de répartir le temps processeur entre les processus (temps partagé).

Une interruption est provoquée par un signal provenant du monde extérieur au processeur, qui modifie temporairement le flux d'exécution du processeur.

ex. fin d'une E/S, arrivée d'un top d'horloge, appui sur une touche clavier, ...

Déni de service - Exemple : UC

```
while(1) { ; }
```

→ Une **interruption** périodique (top d'horloge) permet à l'OS de reprendre la main et de répartir le temps processeur entre les processus (temps partagé).

Une interruption est provoquée par un signal provenant du monde extérieur au processeur, qui modifie temporairement le flux d'exécution du processeur.

ex. fin d'une E/S, arrivée d'un top d'horloge, appui sur une touche clavier, ...

★ **ex. x86** : 32-238. Linux utilise le vecteur 128 (0x80) pour les appels système.

★ Chaque interruption (comme chaque exceptions) dispose d'un **handler** qui prend en charge l'évènement correspondant :

table de vecteurs d'interruptions ou **IDT (Interrupt Descriptor Table)**

Déni de service - Exemple : UC

```
while(1) { ; }
```

→ Une **interruption** périodique (top d'horloge) permet à l'OS de reprendre la main et de répartir le temps processeur entre les processus (temps partagé).

Une interruption est provoquée par un signal provenant du monde extérieur au processeur, qui modifie temporairement le flux d'exécution du processeur.

ex. fin d'une E/S, arrivée d'un top d'horloge, appui sur une touche clavier, ...

★ **ex. x86** : 32-238. Linux utilise le vecteur 128 (0x80) pour les appels système.

★ Chaque interruption (comme chaque exceptions) dispose d'un **handler** qui prend en charge l'évènement correspondant :

table de vecteurs d'interruptions ou **IDT (Interrupt Descriptor Table)**

→ La quantité de ressources accessible doit être limitée :

- **ex.** limiter le temps d'exécution à 60 s dans le shell en cours : `ulimit -t 60`
- effet permanent : `/etc/security/limits.conf`

Déni de service - Exemple : ressource système

```
while(1) { fork(); }
```

→ "fork bomb", une attaque par déni de service - épuiser les ressources système (table des processus) le rendant instable ou inutilisable.

Limitation des ressources :

- `ulimit -u 100` pour limiter le nombre maximal de processus qu'un utilisateur peut créer
- les groupes de contrôle (`cgroups`) permettent de contrôler l'utilisation des ressources par groupe de processus
- `systemd` permet de définir des limites de ressources pour les services système.
ex. ajouter dans le fichier de service d'un démon : `MemoryMax=500M`

Vulnérabilités classiques

2. Escalade de privilèges

- **Principe** : obtenir des droits supérieurs (ex : root) depuis un compte non-privilegié.
- **Vecteurs courants** : bugs du noyau (vulnérabilités de drivers), programmes setuid mal écrits, ...
ex. `passwd` a setuid root pour écrire dans `/etc/shadow`
- **Mécanismes OS associés** :
 - ▶ Contrôle d'accès discrétionnaire / obligatoire (DAC / MAC)
 - ▶ Auditing
- **Contre-mesures** :
 - ▶ Réduire l'usage de setuid; préférer sudo avec logs
 - ▶ Appliquer SELinux/AppArmor, seccomp
 - ▶ Patchs noyau et correctifs de sécurité rapides

Modèles de contrôle d'accès

- **DAC** - Discretionary Access Control :
 - ▶ contrôle basé sur le propriétaire (de la ressource) qui choisit qui peut y accéder **ex.** permissions `chmod`, `chown`
 - ▶ utilisé dans les OS **Linux/UNIX** traditionnels
- **MAC** - Mandatory Access Control :
 - ▶ politique **imposée globalement** par l'administrateur **ex.** SELinux, AppArmor
 - ▶ renforce la **confidentialité** et l'**intégrité**
- **RAC** - Role-based Access Control :
 - ▶ contrôle basé sur les **rôles** attribués aux utilisateurs **ex.** admin, user, auditor
 - ▶ simplifie la gestion dans les environnements **cloud** ou entreprise

Système de protection

Filtrage des appels système - sandboxing

- décider (via des filtres) quels syscalls un processus peut effectuer
→ `seccomp()` (Secure Computing)
- souvent utilisée pour mettre en place des "sandboxes" (environnements d'exécution isolés)
→ réduire la surface d'attaque en cas de compromission d'un programme

Cas d'usage :

- ★ **Navigateurs web** (ex. chrome) : isoler les processus de "rendu" → réduire le risque qu'une faille dans le moteur JavaScript compromise tout le système
- ★ **Conteneurs** (ex. Docker) : renforcer l'isolation des applications - Docker s'exécute directement sur le noyau de l'hôte. Si un conteneur est compromis et qu'il utilise des syscalls critiques (ex. `ptrace`, `mmap`, `reboot`), un attaquant pourrait prendre le contrôle du système hôte.
- ★ **Démons système sensibles** (ex. serveurs comme `ssh`) - un démon s'exécute en continu, souvent avec des privilèges élevés → réduire les actions que l'on peut exécuter limite l'impact d'une éventuelle compromission.

Vulnérabilités classiques

3. Exécution arbitraire

- **Principe** : forcer l'exécution de code choisi par l'attaquant (RCE - Remote Code Execution).
- **Techniques typiques** : buffer overflow / corruption mémoire, injection (commande, SQL, etc.).

Exécution arbitraire : Buffer overflow

```
void fonction(char * chaine) {  
    char buffer[128];  
    strcpy(buffer, chaine);  
}
```

- Famille de failles très répandue (*buffer overflow*)
 - ▶ Récemment: ransomware WannaCry (mai 2017)
- Principe général: faire exécuter du code quelconque à un processus
- Si chaîne fait plus de 128 caractères, alors `strcpy` écrasera l'adresse de retour de la fonction
- Si chaîne contient du code machine, le processus exécutera ce code machine
- Mécanismes de défense :
 - ▶ **W xor X**: interdire qu'une région mémoire soit à la fois inscriptible et exécutable - NX bit (PTE du x86-64)
 - ▶ **ASLR (Address Space Layout Randomization)**: les régions ont des adresses difficiles à prévoir
 - ▶ **canaris** pour détecter les dépassements de pile
 - ▶ En assembleur/C/C++, utiliser des fonctions permettant de traiter des données de manière sécurisée (`strcpy` $\rightsquigarrow$ `strncpy`)
 - ▶ Utiliser des langages de plus haut niveau
 - ▶ Utiliser des outils de test automatiques (*fuzzing*)

Système de protection- La mémoire centrale

✓ Séparation de l'espace d'adressage des différents processus utilisateur ainsi que le noyau → **isolation mémoire**

✓ Cas de la pagination :

- chaque processus dispose de sa propre table des pages,
- cette table est contrôlée par le noyau et gérée par la MMU,
- les droits d'accès (rwx) sont définis par page, si tentative d'accès non autorisée alors **une exception** matérielle déclenchée **ex.**
 - ▶ page du noyau → [x86] General Protection Fault (GPF)
 - ▶ page non présente → [x86] Page Fault (PF)

Système de protection- La mémoire centrale

✓ Séparation de l'espace d'adressage des différents processus utilisateur ainsi que le noyau → **isolation mémoire**

✓ Cas de la pagination :

- chaque processus dispose de sa propre table des pages,
- cette table est contrôlée par le noyau et gérée par la MMU,
- les droits d'accès (rwx) sont définis par page, si tentative d'accès non autorisée alors **une exception** matérielle déclenchée **ex.**
 - ▶ page du noyau → [x86] General Protection Fault (GPF)
 - ▶ page non présente → [x86] Page Fault (PF)

Une exception est un signal provoqué par le processeur lors d'un événement anormal : division par zéro, défaut de page ...

* **x86** : 20 différentes exceptions 0..19.

→ Chaque exception a un gestionnaire dédié (handler) :

- **[abort]** : arrêt du programme fautif
- **[fault]** : **réexécution** de l'instruction en cause.

Vulnérabilités classiques

4. Rootkits - dissimulation de compromissions

- **Principe** : masquer la présence d'un attaquant (processus, fichiers, connexions).
- **Formes possibles**: kernel rootkits (modules noyau, hook system calls), userland rootkits (remplacement d'outils système)
- **Mécanismes OS attaqués** :
 - ▶ Table des syscall / vecteurs d'interruption
 - ▶ Binaire système remplacé **ex.** module noyau qui filtre les entrées de /proc pour masquer un processus - ps est remplacé par un binaire malicieux
 - ▶ ...
- **Détection et atténuation** :
 - ▶ intégrité des fichiers, vérification signatures
 - ▶ Boot sécurisé (Secure Boot) et vérification du MBR/EFI
 - ▶ Forensic : comparaison live vs image propre, journaux réseau, détection d'APIs hookées

Forensic mémoire

- ✓ **Contexte** : la RAM contient l'état volatile (processus, stacks, connexions, tables noyau) - lieu clé pour détecter rootkits et hooks actifs.
- ✓ **Approche générale** :
 - **Acquisition** : capturer une image mémoire fiable (dump)
 - **Analyse** : rechercher processus cachés, hooks de syscall / vecteurs interrupt, modules noyau injectés, etc.
 - **Corrélation** : comparer image live vs image propre & journaux réseau pour confirmer anomalie
- ✓ **Artefacts prioritaires à examiner** :
 - Table des processus et listes liées (/proc vs image mémoire)
 - Table des syscalls / adresses de handlers noyau
 - Modules noyau en mémoire, symboles non signés
 - Stacks et chaînes de commandes (indicateurs d'injection)
 - Sockets ouvertes et connexions réseau actives